

Mission 4 of 4 — Final Integration

Suggested time: ~15 minutes | Deliverable: Final Software Architecture

NimbusCart's leadership has scheduled a final review before the platform goes live for the sale. It's time to bring everything together into one submission-ready architecture.

Final Requirements

- Present one clearly organized final architecture diagram, incorporating every decision made across Missions 1–3.
- Include a short deployment plan: how would you roll this system out to production, and what would trigger a rollback if something went wrong?
- Include a short monitoring plan: what would you watch after launch, and what would count as a warning sign?
- Confirm your final architecture's total cost stays within the 9,500 budget unit cap.

Task: Finalize & Submit

- Review your Architecture Version 2 as a whole — check it still tells one coherent, end-to-end story.
- Clean up and finalize the diagram: remove anything left over from earlier drafts that no longer serves a purpose.
- Write your deployment plan in a few concrete stages (e.g. stage, verify, release, rollback trigger).
- Write your monitoring plan: name the 2–4 signals you'd actually watch, and what threshold would concern you.
- Add up the cost of your final component selections and confirm it fits within the budget cap.
- Write a final justification (3–5 sentences) summarizing your overall engineering approach and the key trade-offs you made.

Deliverable

Your Final Software Architecture: the completed diagram, deployment plan, monitoring plan, a brief cost check, and your final written justification.

Evaluation Focus: Overall Presentation and Organization • Problem Solving and Decision Making

■ Hints — Mission 4 – Final Integration

- Step back and check the whole diagram tells one coherent story before you touch anything else.
- A deployment plan doesn't need to be long — a few concrete stages (e.g. stage, verify, release, rollback trigger) are enough.

- Leave a few minutes at the end purely to check your final cost against the budget cap — it's an easy, avoidable mistake to submit over budget.

■ Hints — General

- There is no single correct architecture. The jury is evaluating your reasoning and clarity, not whether you matched a hidden answer key.
- Re-read the client requirements and business constraints before opening the Technology Inventory — it's easy to pick a popular-sounding option that doesn't actually fit the brief.
- The 60 minutes are split across four missions. Pace yourself; don't spend so long perfecting Mission 1 that later missions get rushed.
- Every deliverable asks for a short written justification somewhere — keep a running note of *why* you chose each component, in plain language.
- Cost and complexity are trade-offs, not penalties. A more expensive or more complex option can be the right call if the requirements justify it — say so.

■ Glossary — terms used on this page

Term	What it means
RTO (Recovery Time Objective)	The maximum acceptable time to restore a system after a failure, before it's considered too slow.
Active-Active Deployment	Running full, independent copies of a system in more than one region at the same time, so both can serve live traffic.
APM (Application Performance Monitoring)	Tooling that tracks how an application is performing in production — response times, errors, and traces through the system.
Budget Unit / Cost Cap	The abstracted monthly cost figure assigned to each technology option in the simulation, used to check choices against the client's spending limit.
Multi-AZ (Multi Availability Zone)	Running infrastructure across more than one physically separate data center location, so a failure in one doesn't take the system down.